Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 238 (2024) 15–23

Procedia Computer Science

www.elsevier.com/locate/procedia

The 15th International Conference on Ambient Systems, Networks and Technologies (ANT)
April 23-25, 2024, Hasselt, Belgium

Automated Extension-Based Penetration Testing for Web Vulnerabilities

Areej Alhogail*, Manal Alkahtani

College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia

Abstract

Depending on manual solutions for finding security vulnerabilities in web applications is time-consuming, error-prone, and requires high levels of expertise. This paper proposes a model for an automated extension-based penetration testing for web vulnerabilities that is compatible with any web application. It consists of three stages that start with information gathering, followed by a vulnerability assessment process to test for exploit. The results are then automatically generated as a report at the conclusion. To examine the model, a tool is developed based on the suggested model. Results demonstrate that the proposed technique has effectively and promptly identified application's vulnerabilities.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs

Keywords: Browser extension, penetration testing, vulnerabilities, web security, application security.

1. Introduction

Every day, hundreds of web applications are released all around the world. Many of these applications process and store sensitive information, and almost all businesses and services, including e-government, e-commerce, banking, and transportation, utilize web applications. However, there are security vulnerabilities in a lot of these applications. Cyberattacks that target web application vulnerabilities have increased along with the release of additional applications.

An application's vulnerability is a flaw or loophole that can be exploited by attackers to gain system privileges [1]. These vulnerabilities may be the result of flawed authentication, incorrect validation, operational flaws in the process, or security misconfigurations. Web application data is vulnerable to attacks and a well-structured method must be found to secure them. Consequently, a scanner that can automatically detect security vulnerabilities in web services is required.

Penetration testing (often referred to as a pen test), is utilized to mitigate risks and potential information loss by identifying existing gaps and providing a comprehensive mitigation plan. It also evaluates the effectiveness of implemented security measures [2]. The primary objective of pen testing is to proactively discover and identify security vulnerabilities before attackers can exploit them [3]. In a pen test, an unauthorized attack is simulated by a user on a test-target system using manually operated tools, automatically programmed tools, or a combination of both, to prevent a web security breach. The results are then applied to address security issues in compliance with accepted

* Corresponding author.
E-mail address: aalhogail@KSU.edu.sa

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs
10.1016/j.procs.2024.05.191

security standards. While manual vulnerability assessment in a web application can be effective, it is also highly time-consuming, error-prone, and costly, thereby limiting its viability and scalability [4].

The growing number of web applications and web servers has led to a need, for automated detection of vulnerabilities. Typically, specialized software is used for web vulnerability scanning. Many of them have limited functionality and don't provide information gathering or exploit files nor do they offer recommendations and deploying the wrong tool for vulnerability detection may result in undiscovered ones [3]. A practical approach for pen testing is using a web browser extension that allows the pen tester to directly modify request headers from their browser. This approach also eliminates the need for separate tools for most pen testing-related tasks. There are a few extensions that support penetration testers in their testing efforts [5], however, tend to be more general in nature and primarily focused on scanning [6]. The literature and practice lack investigation of such tools.

In this study, we propose a model for a comprehensive automated web browser extension, which performs the process of pen tasting, starting from information gathering through to the final report. This model designed to detect vulnerabilities with automated scanning and help developers to know the level of security on the application at a faster rate.

The extension model is designed to be applicable to any web application. Its processes will support pen testers in gathering information about a web application, such as its domain, subdomains, the operating system (OS) that it is running on the server, and the content management system (CMS) platform being deployed. After gathering this information, the second phase is scanning for different common vulnerabilities and explore exploit files. Finally, the proposed model provides reports after each step of the process that contains all the gathered and analysed information. A tool is then developed based on the model to test the model's validity, effectiveness, and performance using unit testing.

The model proposed in this paper also allows developers to conduct security testing during each step of a system's development life cycle, find the weaknesses of an application and provide guidelines on preventing and patching them. It also provides a clear structure for a penetration testing team. The proposed model applies to web applications only, so it does not cover network aspects.

The rest of this paper is organised as follows. In Section 2, a background related to vulnerability detection is presented. A description of the suggested solution is given in Section 3. Section 4, outline the results of the experiments that is used to test the proposed solution. Finally, Section 5 presents the conclusions and suggestions for future study and research.

2. Background

Web applications bring with them new and significant security threats. Each application is different and thus may contain its own unique vulnerabilities. Considering that most applications are developed internally, it is common for developers to have limited knowledge, about security issues, in their code. Several studies have focused on the importance of web application security and have provided a number of approaches to detecting web application vulnerabilities [3-5].

Although there are many vulnerability detection methods available today, many of them are not easy to operate and have unsatisfactory detection accuracy [2 ,7]. A benchmarking strategy was put up by the authors of [8] to evaluate and compare vulnerability detection tool efficacy in Web services. The results show that the suggested benchmarks fairly represent the accuracy of vulnerability detection technologies and recommend that the suggested benchmarking techniques be implemented in the field.

Penetration testing is a type of security assessment used to find risk areas and vulnerabilities that threaten the security of web services [9] to determine how best to patch them and provide improvements, recommendations, and policies to improve overall security. Penetration testers usually employ some standards, such as the Information Systems Security Assessment Framework (ISAAF), Penetration Testing Execution Standard (PTES) and the NIST SP 800-115, to provide guidance for planning and conducting information security testing and assessments [9].

Manual penetration testing usually requires an extensive team of professionals possessing diverse skill sets, which most organisations cannot afford to maintain in-house or contract on a regular basis. Almubairik and Wills [10] stated that the main issue with manual penetration testing is that it leaves a lot of attack space uninvestigated. It can also lead to a failure to identify all of the threats that can affect a system as a single flaw can compromise an entire system. This is not ideal because anyone creating secure web applications should be able to use web application vulnerability scanners. The simple and safe way to perform all tasks related to penetration testing is to use automated methods [11].

Using automated security scans, educating web application developers, and implementing secure coding techniques are all crucial during application development [12].

There are three types of penetration testing methods for finding vulnerabilities: black box testing, white box testing and grey box testing [9]. Black box testing does not require any details or prior knowledge of the target. White box testing provides full access to the target for the penetration tester. Grey box testing involves having partial knowledge of the internal workings of the system being tested, while still treating it as a black box.

The pen tester needs to pass through various stages to complete it. The process consists of six stages: engagement, information gathering, foot printing and scanning, vulnerability assessment, exploitation and reporting, as shown in Figure 1.

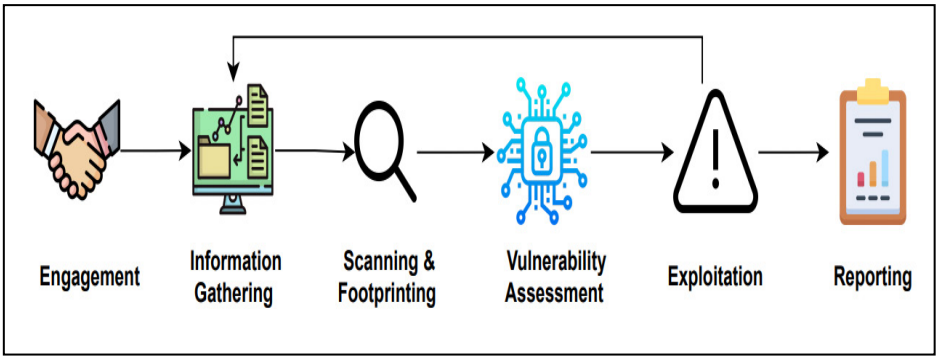


Fig. 1.: Penetration testing life cycle

- Engagement: collect all the details about the penetration test including the type of method to be used, the amount of time the engagement will take, the scope and the complexity of the applications and services and a quote for the cost of the testing.
- Information gathering: to gathers all possible information about the device or system such as IP, subdomains and type of CMS [13].
- Foot printing and scanning: This phase requires deep knowledge of the servers and services to be tested to help penetration testers identify the number of potential vulnerabilities [14]
- Vulnerability assessment: build a list of weaknesses present on the target system. During this phase, the penetration tester analyses the risks identified in the foot printing and scanning process to find the root causes and the effects if the risks were exploited.
- Exploitation: assesses all of the vulnerabilities found during the vulnerability assessment to verifies if any vulnerabilities exist
- Reporting: delivering and communicating the penetration test's results

There are many tools that used in web application penetration testing such as are Nmap [15] to find the web server; Nikto [13] to perform a quick analysis of the web server and applications and report back on a common server and software misconfigurations; and the Metasploit framework [16] to develop and execute exploit code against a remote target machine, testing the computer system for vulnerabilities. Nevertheless, there is still a need for automated comprehensive solutions for web application penetration testing that combine most possible tests.

3. Web Vulnerabilities Penetration Tester (WVPT)

Most browsers have introduced new extension platforms with numerous features intended to prevent and mitigate extension vulnerabilities. These include strong isolation between websites and extensions, privilege separation within an extension and extension-permission systems [17]. Therefore, we proposed a model to provide an extension-based automated process of penetration testing to detect web application vulnerabilities

The proposed solution is illustrated in Figure 2, which gives a schematic view of the web vulnerabilities penetration tester model. The users of the model are penetration testers and developers.

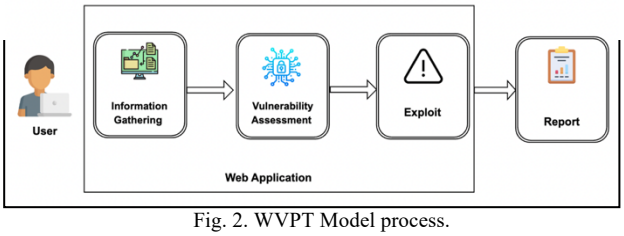


Fig. 2. WVPT Model process.

3.1 Model Process

The model's process consists of three stages to produce comprehensive testing to ensure the effectiveness of web application security. The stages start with collecting all necessary information about the web application, after that the model performs a vulnerability assessment process to test for exploit and then lists the exploit files for these zero-day vulnerabilities. The three stages provide reports after each process. All these phases will be explained in detail in the following sections.

3.1.1 Information Gathering

Information gathering is the first step and basis of a successful penetration test. The more effectively information is gathered about a target web application, the more easily vulnerabilities can be found. Information gathering consists of a search for publicly available information related to the target web application and seeks out the most effective methods of exploiting that information. Figure 3 show the process of information gathering.

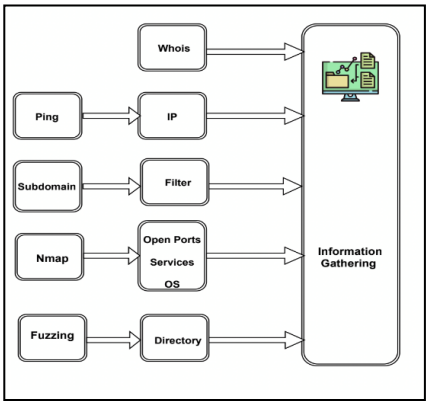


Fig. 3. Information gathering phase.

In this model, there are two approaches to the information gathering process: passive information gathering and active information gathering.

Passive Information Gathering

This approach is typically employed before active information gathering, as it offers general and publicly available information on a web application. It is executed without establishing direct contact between the penetration tester and the target web application. Open-source tools, including whois [18-19] lookup and mail servers, facilitate this method. In this project, a whois tool was integrated with passive information gathering using internet records to obtain public information about the target. This includes details about registered domain names, IP address blocks, server names, and a variety of information services.

Active Information Gathering

Active information gathering involves establishing contact between the penetration tester and the target web application to gather more data. It allows the penetration tester to extract information about open ports, services, versions of applications and versions of the OS, among other elements. As shown in Figure 3, the proposed solution integrated a number of tools to automatically gather information that is utilised in the next stage. The first tool is Ping, which is used to find the IP address of the target web application for use in other tools. The second tool is to locate all

of the subdomains of the target web application using any subdomain scanner tool such as the Sublist3r [20] tool. Additionally, it allows the pen tester to find the vulnerabilities that tend to be present across multiple domains and applications of the same organization.

Third is analysing the services, the OS and open ports on a web application using the Nmap tool. Nmap is a network scanning tool that uses IP packets to actively interacts with target web applications to identify all the devices connected to a network, live hosts, services, open ports and operating systems they are running [15]. Fourth is a fuzzing technique, which is a black-box software testing technique that attempts to locate hidden files and directories on the web server. This technique may expose sensitive files, such as backups, index.php.old and source_code.zip. It can find implementation bugs using malformed/semi-malformed data injection in an automated fashion [21]. This tool uses a dictionary list of all possible directories for testing web applications.

3.1.2 Vulnerability Assessment

A vulnerability assessment provides a systematic review of security weaknesses in an application. It is the process of defining, identifying, classifying, and prioritizing vulnerabilities in the target web application [15]. These assessments are a critical component of penetration testing because they provide details about security weaknesses in the target that lead to the system's exploitation.

This tool conducts vulnerability assessment on well-known and critical vulnerabilities. Additionally, it checks services for any weaknesses or zero-day vulnerabilities. Following this, it provides a report with details on any security weaknesses in its environment. Fig. 4 shows the vulnerabilities assessment in the proposed model, which is as follows:

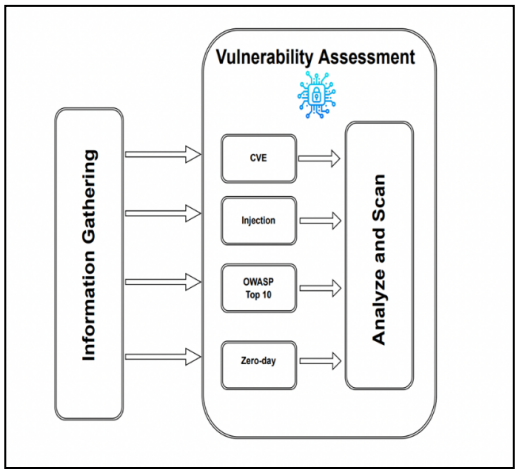


Fig. 4. Vulnerability assessment phase.

- If the model detects any services, the CMS or libraries in the source code, it will automatically search for vulnerabilities in common vulnerabilities and exposures (CVE) [22], which is a database of publicly disclosed information security issues [23]. If there any vulnerabilities, they will be documented to be worked on in the next phase.
- If there are user inputs or fields that allow users to interact with the web application, the model will try to detect injection vulnerabilities, such as structured query language (SQL), cross-site scripting (XSS) and server-side request forgery (SSRF) vulnerabilities, which are the most common.
- The list of 10 most common web application security risks from Open Web Application Security Project top 10 [24] web vulnerabilities, is used in conjunction with an integrated tool to detect whether the target web application contains any security risks.
- The administrator or penetration tester can add zero-day vulnerabilities to the model to keep it up to date.

3.1.3 Exploitation

An exploit is a code that takes advantage of a software weakness or security flaw. An exploit is referred to as a zero-day exploit when it is used to attack a vulnerability that is unidentified and yet to be patched. As numerous zero-day exploits are published every day, they take on many forms, and it is challenging to make the exploit process fully automated. As they require configuration and editing by the pen tester, many exploit scripts are complex and must be

customised so that the parameters are appropriate for the pen tester's machine. For example, if there is an exploit script that requires the identification of the listener to execute it, the pen tester must identify a suitable and unused port. This model provides a report that contains a brief description of the exploit, the severity of exploit, the exploit script, how to execute the script and recommendations on avoiding and patching the exploit.

3.1.4 Reporting

The last, and most important, step of penetration testing is creating a comprehensive report of the results after each process which will guide the pen tester and developer of any detectable vulnerabilities in web applications and permits the developer and managers to patch these weaknesses and improve web application security. It also guides the penetration testing team in checking all existing vulnerabilities that may allow for exploitation of the system.

3.2 Tool Development

The model proposed in this research has been used to guide the development of the Kashef tool, which will be used to automatically detect web vulnerabilities. 'Kashef' is an Arabic name that means 'detector'. This tool will be used to evaluate the accuracy and effectiveness of the proposed model as described in the following section.

The tool's design represents a three-tier concept, separating the system into the presentation tier (web browser extension), the application tier (PHP application) and the data tier (Ubuntu server). Fig. 5 shows the structure of the model.

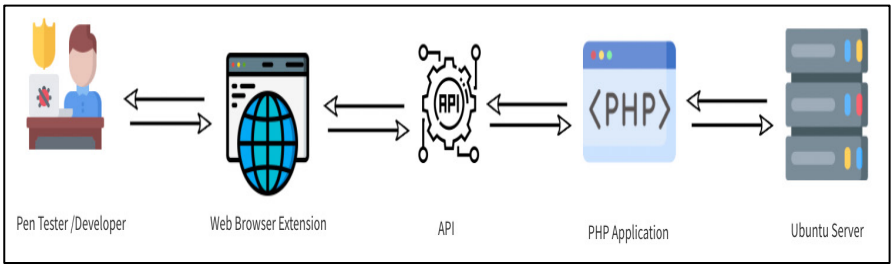


Fig. 5. Tool architecture.

The tool is implemented in a way that allows the pen tester or developer to interact directly with the web browser extension to execute their request in the first stage The second-layer web browser extension will include several functions and deliver each request to the PHP application through API. The third layer is the API, which allows two applications such as the browser extension and the PHP application to talk to each other. In the fourth layer, a simple PHP application that is hosted on an Ubuntu server (the last layer) is used to allow the execution of a command in the command line via an open shell to the server. The same process will be carried out in the opposite direction as a response: the results sent from the server to the PHP application will be fetched, and a report will be created. Then the report will be downloaded via a browser. This tool was developed using several programming languages: JavaScript, PHP, Python, HTML and CSS.

To implement this model, an extension must first be developed. This is the base on which all operations are executed. In this model, we developed a Google Chrome browser extension as it is widely used and provides great functionality in terms of privacy and security. Extensions are made up of a variety of interconnected components [25], including a manifest.json file, background scripts, content scripts, an options page, user interface UI elements and logic files.

4. Experiments

4.1 Testing Environment

The developed tool was tested in the virtual environment *Portswigger lab* that represented a real environment concerned in web security [26]. It provides permissions to test many web applications with several vulnerabilities for educational purposes. The model was tested on a range of vulnerable web applications through a list of vulnerabilities. The purpose of testing was to assess whether the tool could demonstrate its intended functionalities to discover errors and weaknesses through a wide range of metrics, including a good code structure guarantee. The type of testing used

in this model is the unit test, which involves individual methods and functions of the classes and code that can be isolated in a system.

4.2 Results

We applied functional testing to evaluate comprehensive penetration testing processes such as information gathering, vulnerability assessment, exploits and report generation. The testing was successful, and results were accurate. The report generation took less than 40 seconds for information gathering and 30 seconds for vulnerability assessment.

To confirm the results of experiment the tool has been tested in the Vulnhu machines which provide materials that allows anyone to gain practical 'hands-on' experience in digital security, computer software & network administration [27], the result nearly similar the tool discover all information and possible vulnerabilities in a time not exceeding a minute for each stage as it depends on the size of the target web application.

Moreover, non-functional testing provides criteria that can be used to evaluate the operation and functioning of a tool such as automation, performance, real-time, scalability and usability. We compared the tool with a selection of well-known state-of-the-art penetration testing tools such as Open Port Check [28], Wappalyzer [29], FastFuz [30], XSS ChEF [31] and WebSecurify [32]. We found that our tool proved to be comprehensive, accurate, and organised in comparison to the other compared tools. The Kashef tool provided a more comprehensive service in comparison to the other tools. For example, Open Port Check is a Chrome extension that allows the pen tester to check the availability of a port on a network device without accessing the site from the browser [32]. It requires the IP address of the target web application. This is unlike the proposed tool, which only requires an open web application to generate a report with all the necessary information. Wappalyzer is browser extension that shows pen tester what software a website is built with and identify what CMS a web application is using. FastFuz is Chrome extension that allows pen testers to conduct directory fuzzing only.

Vulnerability assessments scan for the most common vulnerabilities to check if a web application has any weaknesses. The proposed tool generated a report with accurate and correct results in a relatively short time of 30 seconds. The report lists all scanning results of XSS, SQL Injections, SSRF and SearchSploit, which allows the pen tester to search for existing services and determine if there are any vulnerabilities or zero-day exploits.

In general, the vulnerability assessment operation used a number of automated penetration testing extensions. XssChEF allows the pen tester to identify XSS vulnerabilities in web applications. It is similar to BeEF but designed for browsers [31]. It doesn't identify SQL Injection or exploit. WebSecurify is a tool that runs directly from the browser, and it is capable of finding XSS and SQL injections [32] but not exploit database. All these tools are customised for specific vulnerabilities and do not cover SSRF vulnerabilities or check other services for potential weaknesses. Moreover, most effective penetration testing tools do not usually provide an exploit section for zero-day vulnerabilities because it is a renewed and difficult field, which requires machine learning for large datasets of exploits. To solve this, the Kashef tool is designed for the user to manually upload any new exploit for detection. It also allows the generation of the results in report form, which can be downloaded automatically.

Using automated extension-based penetration testing for web vulnerabilities is significantly faster, more efficient, easier, and more reliable than manual tests. The proposed tool also allows developers to carry out vulnerability assessments but with more customisation, accuracy and in less time. In regard to non-functional requirements, the tool showed promising results. After testing The Kashef tool on a range of web applications, the results showed that the tool met the performance needs of users by providing many functions for each option in a short time period. Table 1 presents the time required for the testing of each option. In addition, 90% of Kashef's functions are fully automated except for exploit due to the difficulty in specifying the exploit parameters depending on pen testers' individual requirements. For usability, the Kashef tool was tested by a group of pen testers and developers, who all agreed on its ease and clarity of use. The tool Can be viewed at https://github.com/manalsultan/Kashef-toool.

Table 1time to complete each option.

Type	Time to complete	Type	Time to complete	Type	Time to complete	Type	Time to complete
XSS	14 seconds	SQLI	15 seconds	SSRF	5 seconds	Search Sploit	5 seconds

5.Conclusion

In this paper, we proposed a model of automated extension-based penetration testing for web vulnerabilities that provides black box testing to identify cyber security risks that can lead to unauthorized access or data exposure. A tool was developed based on the model to evaluate the validity of the proposed solution. It proved to be able to detect vulnerabilities in a timely manner with comprehensive scanning and minimal manual intervention through its three main stages. First, gather information about web applications such as its domain, subdomains, OS that is running on the server, and the CMS platform it deploys. After that, there is a vulnerability assessment to help the penetration tester scan for common vulnerabilities. Also, the model allows the pen tester to upload and delete exploit files. These files provide a deep analysis of exploit and recommendations on how to mitigate the discovered vulnerabilities and prevent and batch them. The final step is the reporting of all the gathered and analyzed information.

The testing of the developed tool showed promising results. All performed tests gave correct and accurate results within a short period of time. It also outperforms the other well-known penetration tester tools on the market.

For future research, we are planning to utilize machine learning to make the exploit option fully automated. This will analyze the exploit dataset for any services and data, then automatically attempts to exploit it by identifying all requirements, parameters, and options. In addition, the proposed solution should be extended to apply penetration testing over an entire network of websites and servers to become part of an internal network for the latter to be tested.

References

[1] Khera, Y., Kumar, D., & Garg, N. (2019, February). "Analysis and impact of vulnerability assessment and penetration testing". *In 2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon),* pp. 525-530. IEEE.

[2] Altulaihan, E. A., Alismail, A., & Frikha, M., (2023) "A Survey on Web Application Penetration Testing", *Electro*nic, 12(5), 1229.

[3] Albahar, M., Alansari, D., & Jurcut, A. (2022). "An empirical comparison of pen-testing tools for detecting web app vulnerabilities". *Electronics,* 11(19), 2991.

[4] Stefinko, Y., Piskozub, A., & Banakh, R. (2016). "Manual and automated penetration testing. Benefits and drawbacks. Modern tendency". *In 2016 13th international conference on modern problems of radio engineering, telecommunications and computer science (TCSET),* pp. 488-491. IEEE.

[5] Phong, C. T., & Yan, W. Q. (2014). "An overview of penetration testing". *International Journal of Digital Crime and Forensics (IJDCF)*, 6(4), 50-74

[6] Ravindran, U., & Potukuchi, R. V. (2022). "A Review on Web Application Vulnerability Assessment and Penetration Testing". *Review of Computer Engineering Studies,* 9(1).

[7] Alazmi, S., & De Leon, D. C. (2022) . "A systematic literature review on the characteristics and effectiveness of web application vulnerability scanners". *IEEE Access*, 10, 33200-33219.

[8] Antunes, N. and Vieira, M. , (2015) "Assessing and Comparing Vulnerability Detection Tools for Web Services: Benchmarking Approach and Examples," in *IEEE Transactions on Services Computing*, vol. 8, no. 2, pp. 269-283.

[9] Alhamed, M., & Rahman, M. H. (2023). "A Systematic Literature Review on Penetration Testing in Networks: Future Research Directions". *Applied Sciences*, 13(12), 6986.

[10] Almubairik, N. A., & Wills, G. (2016). "Automated penetration testing based on a threat model". *In 2016 11th International Conference for Internet Technology and Secured Transactions (ICITST),* pp. 413-414. IEEE.

[11] Al Shebli, H. M. Z., & Beheshti, B. D. (2018). "A study on penetration testing process and tools". *In 2018 IEEE Long Island Systems, Applications and Technology Conference (LISAT),* pp. 1-7. IEEE.

[12] Baykara, M. (2018). "Investigation and comparison of web application vulnerabilities test tools". *Int. J. Comput. Sci. Mob. Comput.(IJCSMC*), 7(12), 197-212.

[13] Krishnama, S. (2023). "A Process of Penetration Testing Using Various Tools". *Mesopotamian Journal of CyberSecurity,* 2023, 94-104.

[14] Pradeep, I., & Sakthivel, G. (2021). "Ethical hacking and penetration testing for securing us form Hackers". In *Journal of Physics: Conference Series*, 1831(1), p. 012004. IOP Publishing.

[15] Kumar, A. (2023). "Penetration Testing Tools and Techniques". In *Perspectives on Ethical Hacking and Penetration Testing* (pp. 280-306). IGI Global

[16] Rani, S., & Nagpal, R. (2019). "Penetration testing using metasploit framework: an ethical approach". *International Research Journal of Engineering and Technology (IRJET)*, *6*(08).

[17] Eriksson, B., Picazo-Sanchez, P., & Sabelfeld, A. (2022). "Hardening the security analysis of browser extensions". In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing* (pp. 1694-1703).

[18] AlSharaa, B., Thuneibat, S., Masadeh, R., & Alqaisi, M. (2023). "Selected advanced themes in ethical hacking and penetration testing". *Computer Science and Information Technologies*, *4*(1), 69-75.

[19] "What is Whois information and why is it valuable? Help & Support, Domaintools. Available at: https://www.domaintools.com/support/what-is-whois-information-and-why-is-it-valuable. accessed on 16 Nov. 2022.

[20] "Sublist3r". Available at: https://github.com/aboul3la/Sublist3r accessed on 11 November 2023.

[21] Godefroid, P. (2020). "Fuzzing: Hack, art, and science". *Communications of the ACM*, *63*(2), 70-76.

[22] Mell, P., & Grance, T. (2002). "Use of the common vulnerabilities and exposures (cve) vulnerability naming scheme". *NIST Special Publication*, *800*, 51.

[23] Bau, J., Bursztein, E., Gupta, D., & Mitchell, J. (2010, May). State of the art: Automated black-box web application vulnerability testing. *In 2010 IEEE symposium on security and privacy*. pp. 332-345. IEEE.

[24] OWASP.org. "OWASP top ten,". Available: https://owasp.org/www-project-top-ten/ accessed on 16 Nov. 2023.

[25] Chrome Developer. “Develop extensions and theme”. Available: https://developer.chrome.com/docs/extensions/mv3/getstarted/ accessed 11 Nov.2023.
[26] Portswigger.net. “About Portswigger”. Available at: https://portswigger.net/about accessed on 18 Nov. 2023
[27] Vulnhub.com. “About Vulnhub”. Available at https://www.vulnhub.com/about accessed on 17 Jan. 2024
[28] Portscaner.com. “Extension for Google Chrome <Port Cheeker Tool”. Available at: https://portscaner.com/google-chrome-ports-checker-extension accessed on 18 Nov. 2023.
[29] Wappalyzer.com. “Identify technologies on websites”. Available at: https://www.wappalyzer.com/ accessed on 18 Nov. 2023
[30] Kitploit.com. “Fastfuz-Chrome-Ext - Site Fast Fuzzing With Chrome Extension”. Available at: https://amp.kitploit.com/2022/03/fastfuz-chrome-ext-site-fast-fuzzing.html accessed on 18 Nov. 2023
[31] “XSSChef”. Available at: https://github.com/koto/xsschef accessed on 18 Nov. 2023.
[32] Pluginsaddonsextension.com. “Websecurify Chrome Extension”. Available: at https://pluginsaddonsextensions.com/google-chrome/websecurify-chrome-extension_94301 accessed on 18 Nov. 2023